

SQLAlchemy 2.0 ORM Documentation

Official Reference Guide to Relational Mappings, Async Sessions, and Queries

1. SQLAlchemy 2.0 Unified ORM

SQLAlchemy 2.0 unifies Core and ORM components. Models inherit from `DeclarativeBase` with explicit type annotations using ``Mapped[...]` and ``mapped_column()`.

```
from sqlalchemy import String, Integer, ForeignKey
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(50))
    email: Mapped[str] = mapped_column(String(100), unique=True)
    posts: Mapped[list["Post"]] = relationship(back_populates="author")

class Post(Base):
    __tablename__ = "posts"
    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(200))
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
    author: Mapped["User"] = relationship(back_populates="posts")
```

2. Async Engine & Session Execution

SQLAlchemy async engine supports asynchronous drivers like `psycopg/asynccpg` using ``AsyncSession`` and ``select()`` statement construction.

```
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy import select

async def fetch_user_by_email(session: AsyncSession, email: str):
    stmt = select(User).where(User.email == email)
    result = await session.execute(stmt)
    return result.scalar_one_or_none()
```